KSE | Kyiv School of Economics

Applied Linear Algebra

Autumn 2026/2027

LAB 2. APPLICATIONS OF EIGENVECTORS

GENERAL INFORMATION

- It is recommended to use Python and Jupyter Notebook for task 2
- The description of each task indicates a set of recommended libraries and useful links

TASKS

Write a program to perform the following

Task 1: Computing Eigenvalues and Eigenvectors of a Matrix

Write a function that takes a square matrix and returns its eigenvalues and eigenvectors using the NumPy library. Also verify the equality $A \cdot v = \lambda \cdot v$ for each eigenvalue and corresponding eigenvector. The NumPy function `eig` will sometimes return numbers of the complex form a+b*i. As this may return complex values of the form a+0*i, use `np.real` to convert these values into (real) floats.

Video to help visualize the process and recall the necessary theory.

Task 2: Implementation of Image Dimensionality Reduction Using Principal Component Analysis (PCA: Image Compression)

Implement a function to reduce image dimensionality using Principal Component Analysis (PCA).

Required libraries for the task: NumPy, matplotlib, Scikit-learn or their equivalents.

Useful links: brief introduction to PCA, for those who want to understand PCA more deeply 1 and 2, forum discussion link on this topic.

What is PCA?

Principal Component Analysis (PCA) is a technique used in image processing to reduce its dimensionality. In PCA, an image is represented as a matrix of pixel values, and the

algorithm identifies the principal components of the image by finding the eigenvectors of the covariance matrix of pixel values. These eigenvectors represent the most important features or patterns in the image. By reducing the dimensionality of image data using PCA, it becomes easier to compress and store images, as well as perform operations such as image recognition and classification.

Algorithm for task execution

1. Display the original color image and a vector that will contain: image dimensions in pixels and the number of main color channels used

   Example of finding this vector

```
import numpy as np
from matplotlib.image import imread
import matplotlib.pyplot as plt

image_raw = imread("photo.jpg")
print(image_raw.shape)
```

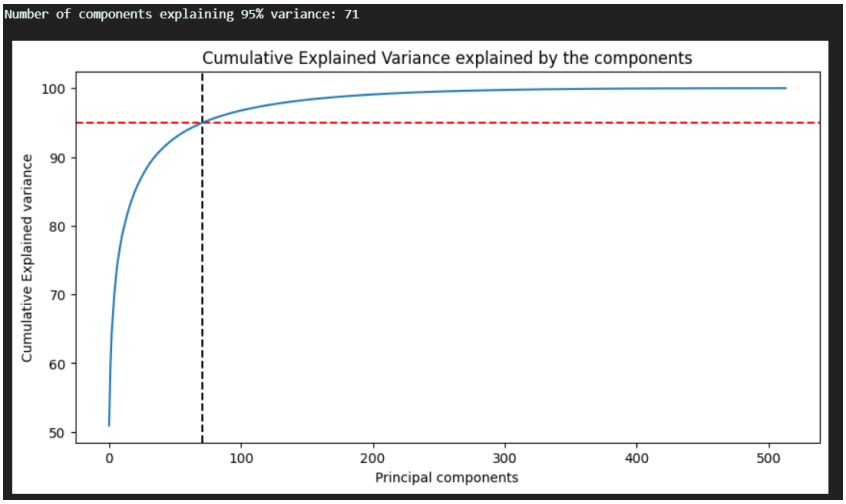
2. Convert the image to black and white and display the image size and number of color channels **(0.25 points)**

```
image_sum = image_raw.sum(axis=2)
print(image_sum.shape)
image_bw = image_sum/image_sum.max()
print(image_bw.max())
```



3. Apply PCA to the image_bw component matrix. Display cumulative variance and find the number of components needed to cover 95% of the variance. Use library tools (NumPy recommended) **(0.75 points)**

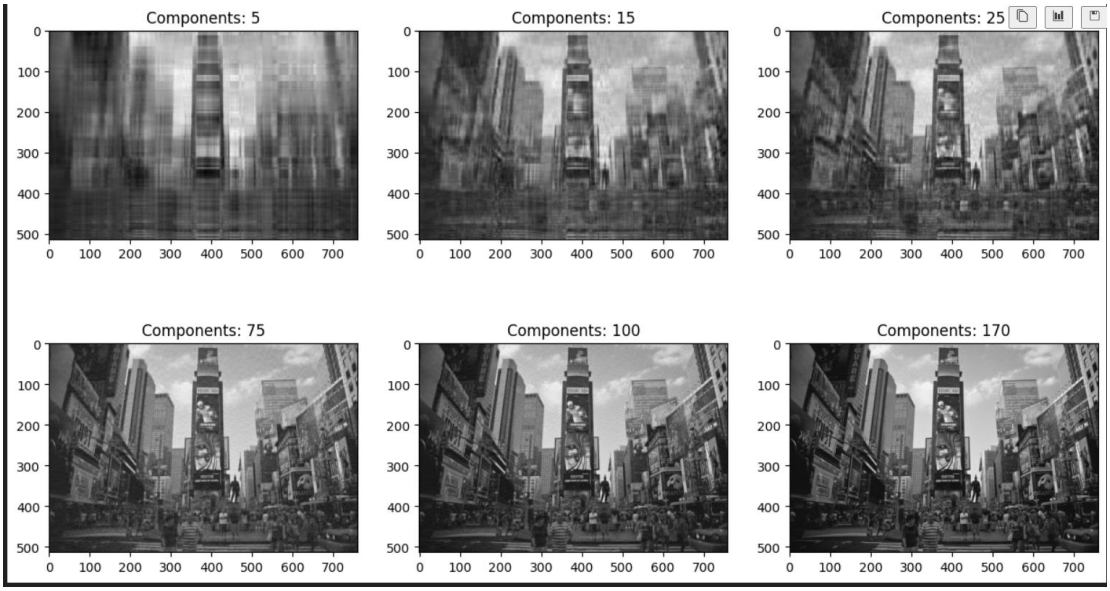
   Display a graph of the corresponding process (**additional 0.5 points)**



   Reconstruct the black and white image using the limited number of components found in the previous step. Display the resulting image. For 95% data coverage, you would expect to get a sharper image, wouldn't you? Note that we have accurately captured all the main elements — you can still identify objects very well. What is missing is sharpness — and perhaps it's the fine details in visual elements that make an image attractive and sharp. **1 point**



4. Reconstruct the image for different numbers of components and display the corresponding results. Try taking a larger number of components and display the corresponding result. Did you get a sharper image? What if you take fewer components? **(2 points)**



Task 3: Using Diagonalization, Eigenvalues and Eigenvectors in Cryptography
Use diagonalization to decrypt codes. It is recommended to use the NumPy library.
Useful links: Brief description of how linear algebra principles can be applied in cryptography, video about cryptography itself

1. Create a function `decrypt_message_(encrypted_vector, key_matrix)` that decrypts the encrypted vector `encrypted_vector` using the key matrix `key_matrix`, using the inverse diagonalization operation.
   The encoding function is defined as follows

```
def encrypt_message(message, key_matrix):

    message_vector = np.array([ord(char) for char in message])
    eigenvalues, eigenvectors = np.linalg.eig(key_matrix)
    diagonalized_key_matrix = np.dot(np.dot(eigenvectors, np.diag(eigenvalues)), np.linalg.inv(eigenvectors))
    encrypted_vector = np.dot(diagonalized_key_matrix, message_vector)
    return encrypted_vector
```

2. Verify the operation of the developed functions using an example of a randomly generated key matrix and text message

```python
# Генерація матриці ключа (приклад)
key_matrix = np.random.randint(0, 256, (len(message), len(message)))
```

Example execution

- Original Message: Hello, World!
- Encrypted Message: [118703.+1.04957181e-11j 180926.+1.50897913e-11j 149312.+1.90968912e-11j 161873.+1.52982299e-11j 188078.+1.70487245e-12j 145036.+1.64536004e-11j 139370.+4.15241913e-11j 195037.+1.51003306e-11j 155629.+1.38144125e-11j 206859.+1.38439864e-11j 145588.+1.95891393e-11j 130163.+1.55598216e-11j 176423.+1.26512259e-11j]
- Decrypted Message: Hello, World!

LAB GRADING

Correctness/quality of the code 6 points
Task 1 – 0.5 points
Task 2 – 4 points
Task 3 – 1.5 points
The maximum mark for each assignment is given if the program works correctly and the results of its work fully meet the requirements of the task.

Laboratory work defense 4 points
The maximum mark is given if you can explain the results of each program, analyze them, and draw appropriate conclusions.

Theoretical Questions 2 points

1. 0.25pts What are eigenvalues and eigenvectors of a matrix? How are they computed?
2. 0.25pts What properties do eigenvectors of symmetric matrices have?
3. 0.75pts What can be the disadvantages of using PCA, and what strategies can be used to overcome these disadvantages?
4. 0.75pts What advantages does matrix diagonalization have in cryptography? How is it applied for encryption and decryption of messages?